

COURSE CODE/TITLE:	CPE010	SECTION:	CPE21S1
DATE PERFORMED:	21/07/26	DATE SUBMITTED:	27/07/26
NAME:	Duenas Ralph Florenz F.	INSTRUCTOR:	Engr. Jimlord Quejado

ACTIVITY NO. ____ [ACTIVITY NAME]

1. PROCEDURE OUTPUT

What procedures did you perform?

- 1.

main.cpp

```
#include <iostream>
#include "singly_ll.h"

class Node{
public:
    char data;
    Node* next;
};

int main (){
    Node*head = nullptr;
    Node*second = nullptr;
    Node*third = nullptr;
    Node*fourth = nullptr;
    Node*fifth = nullptr;

    head = new Node;
    second = new Node;
    third = new Node;
    fourth = new Node;
    fifth = new Node;
```

```
head -> data = 'J';
head -> next = second;

second -> data = 'I';
second -> next = third;

third -> data = 'R';
third -> next = fourth;

fourth -> data = 'Q';
fourth -> next = fifth;

fifth -> data = 'H';
fifth -> next = nullptr;

delete head;
delete second;
delete third;
delete fourth;
delete fifth;

return 0;
```

C singly_ll.h •

```
#ifndef SINGLY_LL_H
#define SINGLY_LL_H

template <typename T>
class SingleList{
public:
    T data;
    singleList<T> *next = nullptr;
};

#endif
```

- 2.

```

1 #include <iostream>
2 #include "singly_ll.h"
3
4 int main() {
5     SingleList<int> *head = new SingleList<int>;
6     SingleList<int> *second = new SingleList<int>;
7     SingleList<int> *third = new SingleList<int>;
8
9     // storing the data and linking the nodes
10    head->data = 1;
11    head->next = second;
12
13    second->data = 2;
14    second->next = third;
15
16    third->data = 3;
17    third->next = nullptr;
18
19    std::cout << "Linked list traversal: ";
20    listTraversal(head);
21
22    delete head;
23    delete second;
24    delete third;
25
26    return 0;
27 }
```

```

1 #ifndef SINGLY_LL_H // header filename
2 #include <iostream> // Included so std::cout and std::endl
3
4 template <typename T> // allows the class to accept any data type
5 class SingleList {
6 public:
7     T data; // data of the node
8     SingleList<T> *next = nullptr; // stores address of the next node
9 };
10
11 template <typename T>
12 void listTraversal(SingleList<T> *head) {
13     while (head != nullptr) {
14         std::cout << head->data;
15
16         // checks if the next node is not equal to null
17         if (head->next != nullptr) {
18             std::cout << "->";
19         }
20         head = head->next;
21     }
22     std::cout << std::endl;
23 }
24
25 #endif
```

Linked list traversal: 1->2->3

- 3.

```
1  ~ #include <iostream>
2  ~ #include "singly_ll.h"
3
4  ~ int main() {
5      ~     SingleList<int> *head = new SingleList<int>;
6      ~     SingleList<int> *second = new SingleList<int>;
7      ~     SingleList<int> *third = new SingleList<int>;
8
9      ~     // storing the data and linking the nodes
10     ~     head->data = 1;
11     ~     head->next = second;
12
13     ~     second->data = 2;
14     ~     second->next = third;
15
16     ~     third->data = 3;
17     ~     third->next = nullptr;
18
19     ~     std::cout << "Linked list traversal: ";
20     ~     listTraversal(head);
21
22     ~     std::cout << "Newly Inserted Head: ";
23     ~     sllinsertHead(0, &head);
24     ~     listTraversal(head);
25
26
27     ~     delete head;
28     ~     delete second;
29     ~     delete third;
30
31     ~     return 0;
32 }
```

```
#ifndef SINGLY_LL_H // header filename
#define SINGLY_LL_H

#include <iostream> // Included so std::cout and std::endl work

template <typename T> // allows the class to accept any data type
class SingleList {
public:
    T data; // data of the node
    SingleList<T> *next = nullptr; // stores address of the next node
};

template <typename T>
void listTraversal(SingleList<T> *head) {
    while (head != nullptr) {
        std::cout << head->data;

        // checks if the next node is not equal to null
        if (head->next != nullptr) {
            std::cout << "->";
        }
        head = head->next;
    }
    std::cout << std::endl;
}

template <typename T>
void sllinsertHead(T newData, SingleList<T>*& currentHead){

    SingleList<T>* newNode = new SingleList<T>;

    newNode->data = newData;

    newNode->next = *currentHead;

    *currentHead = newNode;
}

template <typename T>
void sllGeneralInsertion(T newdata, SingleList<T> *prevNode){

    if(prevNode==nullptr){
        std::cout<< "Previous node cannot be null." << std::endl;
        return;
    }

    SingleList<T>* newNode = new SingleList<T>;

    newNode->data = newdata;

    newNode->next = prevNode->next;

    prevNode->next = newNode;
}

#endif
```

- 4.

```

1  #include <iostream>
2  #include "singly_ll.h"
3
4
5  int main() {
6      // this line creates a empty linked list
7      SingleList<int> *head = new SingleList<int>;
8      SingleList<int> *second = new SingleList<int>;
9      SingleList<int> *third = new SingleList<int>;
10
11
12      head -> data = 1;
13      head -> next = second;
14
15
16      head -> data = 2;
17      head -> next = third;
18
19
20      head -> data = 3;
21      head -> next = nullptr;
22
23
24      std::cout << "Linked List Traversal: ";
25      listTraversal(head);
26
27
28      std::cout << "Newly Inserted Head: ";
29      sllinsertHead(0, &head);
30      listTraversal(head);
31
32
33      std::cout << "Newly inserted after a node of linked list: \n";
34      sllGeneralInsertion(3, head->next);
35      listTraversal(head);
36
37
38      sllGeneralInsertion(4, head->next->next);
39      listTraversal(head);
40
41
42      sllinsertEnd(10, &head);
43      listTraversal(head);
44
45
46      delete head;
47      delete second;
48      delete third;
49
50
51      return 0;
52  }
```

```

1  #ifndef SINGLY_LL_H // header filename
2  #define SINGLY_LL_H
3
4  #include <iostream> // Included so std::cout and std::endl work
5
6  template <typename T> // allows the class to accept any data type
7  class SingleList {
8  public:
9      T data; // data of the node
10     SingleList<T> *next = nullptr; // stores address of the next node
11 };
12
13 template <typename T>
14 void listTraversal(SingleList<T> *head) {
15     while (head != nullptr) {
16         std::cout << head->data;
17
18         // checks if the next node is not equal to null
19         if (head->next != nullptr) {
20             std::cout << "->";
21         }
22         head = head->next;
23     }
24     std::cout << std::endl;
25 }
26
27 template <typename T>
28 void sllinsertHead(T newData, SingleList<T>*& currentHead){
29
30     SingleList<T>* newNode = new SingleList<T>;
31
32     newNode->data = newData;
33
34     newNode->next = *currentHead;
35
36     *currentHead = newNode;
37 }
38
39 template <typename T>
40 void sllGeneralInsertion(T newdata, SingleList<T> *prevNode){
41
42     if(prevNode==nullptr){
43         std::cout<< "Previous node cannot be null." << std::endl;
44         return;
45     }
46
47     SingleList<T>* newNode = new SingleList<T>;
48
49     newNode->data = newdata;
50
51     newNode->next = prevNode->next;
52
53     prevNode->next = newNode;
54
55 }

```

```
template <typename T>
void sllinsertEnd(T newData, SingleList<T>** head){

    SingleList<T>* currentNode = *head;

    //Allocate new node
    SingleList<T>* newNode = new SingleList<T>;

    //Dereference to the head node
    SingleList<T>* currentNode = *head;

    // Store data in new node
    newNode->data = newData;
    //Point next of new node to NULL
    newNode->next = nullptr;

    // check if the list is empty
    if(currentNode == nullptr){
        currentNode = newNode;
        return;
    }

    //Traverse the list until next of the node is null
    while(currentNode->next != nullptr){
        //iteration goint to one node to another
        currentNode = currentNode->next;
    }

    currentNode ->next = newNode;
}

#endif
```

- 5.

```
1 #include <iostream>
2 #include "singly_ll.h"
3
4 int main() {
5     // this line creates a empty linked list
6     singlelist<int> *head = new singlelist<int>;
7     singlelist<int> *second = new singlelist<int>;
8     singlelist<int> *third = new singlelist<int>;
9
10
11     head -> data = 1;
12     head -> next = second;
13
14
15     head -> data = 2;
16     head -> next = third;
17
18
19     head -> data = 3;
20     head -> next = nullptr;
21
22
23     std::cout << "Linked List Traversal: ";
24     listTraversal(head);
25
26
27     std::cout << "Newly Inserted Head: ";
28     sllinsertHead(0, &head);
29     listTraversal(head);
30
31
32     std::cout << "Newly inserted after a node of linked list: \n";
33     sllGeneralInsertion(3, head->next);
34     listTraversal(head);
35
36
37     sllGeneralInsertion(4, head->next->next);
38     listTraversal(head);
39
40
41     sllinsertEnd(10, &head);
42     listTraversal(head);
43
44
45     delete head;
46     delete second;
47     delete third;
48
49
50     return 0;
51 }
```

```
2 #ifndef SINGLY_LL_H // header file name
3 #define SINGLY_LL_H
4 #include<iostream>
5 template <typename T> // enables the class accept any data type
6
7
8 class singlelist{
9 public:
10     T data; //data of the node
11     singlelist<T> *next = nullptr; //stores address of next node
12
13
14 };
15 template <typename T>
16 void listTraversal(singlelist<T>* head){
17
18
19     while( head != nullptr){
20         std::cout << head-> data;
21
22
23         //to check if the next line is not equal to null
24         if (head-> next != nullptr){
25             std::cout << "->";
26         }
27         head = head ->next;
28     }
29
30
31     std::cout << std::endl;
32 }
33
34
35 template <typename T>
36 void sllinsertHead(T newData, singlelist<T>** currentHead){
37
38
39     //allocate the memory for the new node
40     singlelist<T>* newNode = new singlelist<T>;
41     // put new data in the new node
42     newNode ->data = newData;
43     //set next of the new node to point to the previous head
44     newNode -> next = *currentHead;
45     //reset head to point to the new node
46     *currentHead = newNode;
47 }
48
```

```
template <typename T>
void sllGeneralInsertion(T newData, singlelist<T> * prevNode){

    //check if it is the head node (previous node is null)
    if(prevNode==nullptr){
        std::cout <<"Previous node cannot be null." << std::endl;
        return;
    }

    //allocate data
    singlelist<T>* newNode = new singlelist<T>;
    //store data in new node
    newNode -> data = newData;
    //point new node to the previous node was pointing to
    newNode -> next = prevNode -> next;
    //point previous node to the new
    prevNode -> next = newNode;
}

template <typename T>
void sllinsertEnd(T newData, singlelist<T>*& head){

    //dereference to the head node
    singlelist<T>* currentNode = *head;

    //allocate new node
    singlelist<T>* newNode = new singlelist<T>;

    //store data in new node
    newNode ->data = newData;

    //point next of new node to null
    newNode-> next = nullptr;

    //check if the list is empty

    if(currentNode == nullptr){
        currentNode == newNode;
        return;
    }
}
```

```
//traverse the list until next of the node is null
while(currentNode-> next != nullptr){
    //iteration foing to one node to another
    currentNode = currentNode -> next;
}
currentNode -> next = newNode;
}

template <typename T>
void sllDelete(T findData, singlelist<T>*& head){

//check if there is a list
if(*head == nullptr) return;

singlelist<T>* currNode = *head;
singlelist<T>* prevNode = nullptr;

//search for the node
while(currNode != nullptr && currNode->data != findData){
    prevNode = currNode;
    currNode = currNode->next;
}

//if data is not found
if(currNode == nullptr)return;

//delete the head node (the headnode is the find dara)
if (prevNode == nullptr){
    *head = currNode ->next;

}else{

    prevNode -> next = currNode -> next;
}

delete currNode;
}

#endif
```

```
Linked List Traversal: 3
Newly Inserted Head: 0->3
Newly inserted after a node of linked list:
0->3->3
0->3->3->4
0->3->3->4->10
```

- 6.

```
1  #include <iostream>
2  #include "singly_ll.h"
3
4
5
6
7  int main() {
8      // this line creates a empty linked list
9      singlelist<int> *head = new singlelist<int>;
10     singlelist<int> *second = new singlelist<int>;
11     singlelist<int> *third = new singlelist<int>;
12
13
14     head -> data = 1;
15     head -> next = second;
16
17
18     head -> data = 2;
19     head -> next = third;
20
21
22     head -> data = 3;
23     head -> next = nullptr;
24
25
26     std::cout << "Linked List Traversal: ";
27     listTraversal(head);
28
29
30     std::cout << "Newly Inserted Head: ";
31     sllinsertHead(0, &head);
32     listTraversal(head);
33
34
35     std::cout << "Newly inserted after a node of linked list: \n";
36     sllGeneralInsertion(3, head->next);
37     listTraversal(head);
38
39
40     sllGeneralInsertion(4, head->next->next);
41     listTraversal(head);
42
43
44     sllinsertEnd(10, &head);
45     listTraversal(head);
46
47
48     std::cout << "deleting a node: \n";
49     sllDelete(10, &head);
50     listTraversal(head);
51
52
53     std::cout << "Deleting the list \n";
54     listTraversal(head);
55     sllDeleteList(&head);
56     listTraversal(head);
57
58
59
60     return 0;
61 }
```

```

1  #ifndef SINGLY_LL_H // header file name
2  #define SINGLY_LL_H
3  #include<iostream>
4  template <typename T> // enables the class accept any data type
5
6
7  class singlelist{
8  public:
9      T data; //data of the node
10     singlelist<T> *next = nullptr; //stores address of next node
11
12
13 };
14 template <typename T>
15 void listTraversal(singlelist<T>* head){
16
17
18     while( head != nullptr){
19         std::cout << head-> data;
20
21
22         //to check if the next line is not equal to null
23         if (head-> next != nullptr){
24             std::cout << "->";
25         }
26         head = head ->next;
27     }
28
29
30     std::cout << std::endl;
31 }
32
33
34 template <typename T>
35 void sllinsertHead(T newData, singlelist<T>** currentHead){
36
37
38     //allocate the memory for the new node
39     singlelist<T>* newNode = new singlelist<T>;
40     // put new data in the new node
41     newNode ->data = newData;
42     //set next of the new node to point to the previous head
43     newNode -> next = *currentHead;
44     //reset head tp point to the new node
45     *currentHead = newNode;
46 }
47
48
49 template <typename T>
50 void sllGeneralInsertion(T newData, singlelist<T> * prevNode){
51
52
53     //check if it is the head node (previous node is null)
54     if(prevNode==nullptr){
55         std::cout <<"Previous node cannot be null." << std::endl;
56         return;
57     }

```

```

60 //allocate data
61 singlelist<T>* newNode = new singlelist<T>;
62 //store data in new node
63 newNode -> data = newData;
64 //point nre node to to the previous node was pointing to
65 newNode -> next = prevNode -> next;
66 //point previous node to the new
67 prevNode -> next = newNode;
68 }
69
70
71 template <typename T>
72 void sllinsertEnd(T newData, singlelist<T>** head){
73
74
75 //dereference to the head node
76 singlelist<T>* currentNode = *head;
77
78
79 //allocate new node
80 singlelist<T>* newNode = new singlelist<T>;
81
82
83 //store data in new node
84 newNode ->data = newData;
85
86
87 //point next of new node to null
88 newNode-> next = nullptr;
89
90
91 //check if the list is emptu
92
93
94 if(currentNode == nullptr){
95     currentNode == newNode;
96     return;
97 }
98
99
100
101
102 //traverse the list until nxt of the node is null
103 while(currentNode-> next != nullptr){
104     //iteration foing to one node to another
105     currentNode = currentNode -> next;
106 }
107 currentNode -> next = newNode;
108 }
109
110
111 template <typename T>
112 void sllDelete(T findData, singlelist<T>** head){
113
114 //check if there is a list
115 if(*head == nullptr) return;
116
117
118 singlelist<T>* currNode = *head;
119 singlelist<T>* prevNode = nullptr;

```

```
122 //search for the node
123 while(currNode != nullptr && currNode->data != findData){
124     prevNode = currNode;
125     currNode = currNode->next;
126
127
128 }
129
130
131 //if data is not found
132 if(currNode == nullptr) return;
133
134
135 //delete the head node (the headnode is the find data)
136 if (prevNode == nullptr){
137     *head = currNode ->next;
138
139
140 }else{
141
142     prevNode -> next = currNode -> next;
143 }
144
145
146 delete currNode;
147 }
148
149
150
151 template <typename T>
152
153
154 void sllDeleteList(singlelist<T> ** head){
155     singlelist<T>* currentNode = *head;
156
157
158     while (currentNode != nullptr){
159         singlelist<T>* temp = currentNode;
160         currentNode = currentNode->next;
161         delete temp;
162     }
163
164
165
166     *head = nullptr;
167 }
168
169
170 #endif
```

```
Linked List Traversal: 3
Newly Inserted Head: 0->3
Newly inserted after a node of linked list:
0->3->3
0->3->3->4
0->3->3->4->10
```

- 7.

```
1  #include <iostream>
2  #include "singly_LL.h"
3  #include "double_LL.h"
4
5  int main() {
6
7      // 'C' 'P' 'E'
8      // CREATION OF HEAD NODE:
9      doubleList<char>* head = new_node('C');
10     // CREATION OF OTHER NODES:
11     doubleList<char>* second = new_node('P');
12     doubleList<char>* third = new_node('E');
13
14     // LINK THE NODES
15     head->next = second;
16     second->prev = head;
17
18     second->next = third;
19     third->prev = second;
20
21     std::cout << "Initial Traversal" << std::endl;
22     dllTraverse(head);
23
24     std::cout << "Inserting at the head" << std::endl;
25     dllInsertHead('G', &head);
26     dllTraverse(head);
27
28     // Demonstrating General Insertion (Insert 'X' after 'P')
29     std::cout << "General Insertion (Inserting 'X' after 'P'):" << std::endl;
30     dllInsertAfter(second, 'X');
31     dllTraverse(head);
32
33     // Demonstrating Insertion at End (Insert 'Z' at the end)
34     std::cout << "Inserting at the end ('Z'):" << std::endl;
35     dllInsertEnd('Z', head);
36     dllTraverse(head);
37
38     // Demonstrating General Deletion (Deleting 'X')
39     std::cout << "General Deletion (Deleting 'X'):" << std::endl;
40     dllDeleteNode(&head, second->next);
41     dllTraverse(head);
42
43     // Demonstrating Deletion from End
44     std::cout << "Deleting from the end:" << std::endl;
45     dllDeleteEnd(&head);
46     dllTraverse(head);
47
48     return 0;
49 }
```

```

101 // =====
102 // insertion at the end
103 // =====
104 template <typename T>
105 void dllInsertEnd(T newData, doubleList<T>* currentHead) {
106     // create a new node
107     doubleList<T>* newNode = new_node(newData);
108
109     if (currentHead == nullptr) {
110         return;
111     }
112
113     // traverse until the last node
114     doubleList<T>* currentNode = currentHead;
115     while (currentNode->next != nullptr) {
116         currentNode = currentNode->next;
117     }
118
119     // connect the last node to the new node
120     currentNode->next = newNode;
121
122     // connect the prev node to the new node
123     newNode->prev = currentNode;
124 }
125
126 // =====
127 // deletion
128 // =====
129
130 // delete from head
131 template <typename T>
132 void dllDeleteHead(doubleList<T>** currentHead) {
133     if (*currentHead == nullptr) return;
134
135     doubleList<T>* temp = *currentHead;
136     *currentHead = (*currentHead)->next;
137
138     if (*currentHead != nullptr) {
139         (*currentHead)->prev = nullptr;
140     }
141
142     delete temp;
143 }

```

```
// general deletion (deletes a given node)
template <typename T>
void dllDeleteNode(doubleList<T>*& currentHead, doubleList<T>* delNode) {
    if (*currentHead == nullptr || delNode == nullptr) return;

    // if node to be deleted is head node
    if (*currentHead == delNode) {
        *currentHead = delNode->next;
    }

    // change next only if node to be deleted is NOT the last node
    if (delNode->next != nullptr) {
        delNode->next->prev = delNode->prev;
    }

    // change prev only if node to be deleted is NOT the first node
    if (delNode->prev != nullptr) {
        delNode->prev->next = delNode->next;
    }

    delete delNode;
}

// delete from end
template <typename T>
void dllDeleteEnd(doubleList<T>*& currentHead) {
    if (*currentHead == nullptr) return;

    doubleList<T>* temp = *currentHead;

    if (temp->next == nullptr) {
        *currentHead = nullptr;
        delete temp;
        return;
    }

    while (temp->next != nullptr) {
        temp = temp->next;
    }

    temp->prev->next = nullptr;
    delete temp;
}

#endif
```

- LESSON LEARNED

Based on my understanding of the code I worked on, I learned a lot about how programming works behind the scenes. One thing I realized is that programming is very strict with uppercase and lowercase letters. Even a small difference in capitalization can cause errors and prevent the program from running correctly. I also learned how some functions can directly change the original data by using references, which helped me understand how different parts of a program can work together. Another important lesson for me was that I need to be responsible for managing the memory I use. If I create something new in my program, I should also make sure to remove it when it is no longer needed to avoid wasting memory. Lastly, I learned that adding safety checks before accessing data is very important because it helps prevent errors and keeps my program from crashing. Overall, this experience helped me become more careful, patient, and attentive to small details while writing code.

- Analysis Of the Procedure

In the main procedure (main.cpp), the program initialized nodes containing characters ('C', 'P', 'E') using `new_node()` and manually linked them using their next and prev pointers. The program then used `dllTraverse()` to print the list in forward order and reverse order using a tail pointer. Next, it tested insertion functions: `dllInsertHead()` inserted 'G' at the start, `dllInsertAfter()` added 'X' after 'P', and `dllInsertEnd()` attached 'Z' to the end of the list. Lastly, deletion was demonstrated using `dllDeleteNode()` to un-link and deallocate middle node 'X', and `dllDeleteEnd()` to remove the last node 'Z', updating adjacent pointers accordingly.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

```

1  #include <iostream>
2  #include <string>
3  #include "double_LL.h"
4
5  int main()
6
7      // CREATION OF HEAD NODE:
8      doubleList<std::string>* head = new_node(std::string("Song_A"));
9
10     // CREATION OF OTHER NODES:
11     doubleList<std::string>* second = new_node(std::string("Song_B"));
12     doubleList<std::string>* third = new_node(std::string("Song_C"));
13
14     // LINK THE NODES
15     head->next = second;
16     second->prev = head;
17
18     second->next = third;
19     third->prev = second;
20
21     std::cout << "Initial Traversal" << std::endl;
22     dllTraverse(head);
23
24     std::cout << "Inserting at the head" << std::endl;
25     dllInsertHead(std::string("Song_Intro"), &head);
26     dllTraverse(head);
27
28     std::cout << "General Insertion (Inserting 'Song_X' after 'Song_B'):" << std::endl;
29     dllInsertAfter(second, std::string("Song_X"));
30     dllTraverse(head);
31
32     std::cout << "Inserting at the end ('Song_Z'):" << std::endl;
33     dllInsertEnd(std::string("Song_Z"), head);
34     dllTraverse(head);
35
36     std::cout << "General Deletion (Deleting 'Song_X'):" << std::endl;
37     dllDeleteNode(&head, second->next);
38     dllTraverse(head);
39
40     std::cout << "Deleting from the end:" << std::endl;
41     dllDeleteEnd(&head);
42     dllTraverse(head);
43
44     return 0;
45

```

```
1  #ifndef DOUBLE_LL_H
2  #define DOUBLE_LL_H
3
4  #include <iostream>
5  #include <string>
6
7  template <typename T>
8  class doubleList {
9  public:
10     T data;
11     doubleList<T>* next = nullptr;
12     doubleList<T>* prev = nullptr;
13 };
14
15 // =====
16 // creating a node
17 // =====
18 template <typename T>
19 doubleList<T>* new_node(T newData) {
20     doubleList<T>* newNode = new doubleList<T>();
21     newNode->data = newData;
22     newNode->next = nullptr;
23     newNode->prev = nullptr;
24     return newNode;
25 }
26
27 // =====
28 // traversal
29 // =====
30 template <typename T>
31 void dllTraverse(doubleList<T>* currentNode) {
32     doubleList<T>* tail = nullptr;
33     while (currentNode != nullptr) {
34         // prints the data
35         std::cout << currentNode->data << " ";
36         // save the current node as the last visited node
37         tail = currentNode;
38         // move to the next node
39         currentNode = currentNode->next;
40     }
41
42     std::cout << std::endl;
43
44     // reverse traversal
45     std::cout << "Reverse Traversal: ";
46     while (tail != nullptr) {
47         std::cout << tail->data << " ";
48         tail = tail->prev;
49     }
```

```
50
51     std::cout << std::endl;
52 }
53
54 // =====
55 // insertion at the head
56 // =====
57 template <typename T>
58 void dllInsertHead(T newData, doubleList<T>*& currentHead) {
59     // create a new node
60     doubleList<T>* newNode = new_node(newData);
61
62     // SET the new node to point to the current head
63     newNode->next = *currentHead;
64
65     // current head will be pointed to the new node
66     if (*currentHead != nullptr) {
67         (*currentHead)->prev = newNode;
68     }
69
70     // update the head pointer
71     *currentHead = newNode;
72 }
73
74 // =====
75 // general insertion
76 // =====
77 template <typename T>
78 void dllInsertAfter(doubleList<T>* prevNode, T newData) {
79     if (prevNode == nullptr) {
80         std::cout << "The given previous node cannot be NULL" << std::endl;
81         return;
82     }
83
84     // create a new node
85     doubleList<T>* newNode = new_node(newData);
86
87     // set new node's next to prevNode's next
88     newNode->next = prevNode->next;
89
90     // connect prevNode's next to newNode
91     prevNode->next = newNode;
92
93     // connect newNode's prev to prevNode
94     newNode->prev = prevNode;
```

```

96     // connect next node's prev to newNode if it exists
97     if (newNode->next != nullptr) {
98         |     newNode->next->prev = newNode;
99     }
100 }
101
102 // =====
103 // insertion at the end
104 // =====
105 template <typename T>
106 void dllInsertEnd(T newData, doubleList<T>* currentHead) {
107     // create a new node
108     doubleList<T>* newNode = new_node(newData);
109
110     if (currentHead == nullptr) {
111         |     return;
112     }
113
114     // traverse until the last node
115     doubleList<T>* currentNode = currentHead;
116     while (currentNode->next != nullptr) {
117         |     currentNode = currentNode->next;
118     }
119
120     // connect the last node to the new node
121     currentNode->next = newNode;
122
123     // connect the prev node to the new node
124     newNode->prev = currentNode;
125 }
126
127 // =====
128 // general deletion (deletes a given node)
129 // =====
130 template <typename T>
131 void dllDeleteNode(doubleList<T>*& currentHead, doubleList<T>* delNode) {
132     if (*currentHead == nullptr || delNode == nullptr) return;
133
134     // if node to be deleted is head node
135     *if (*currentHead == delNode) {
136         |     *currentHead = delNode->next;
137     }
138
139     // change next only if node to be deleted is NOT the last node
140     if (delNode->next != nullptr) {
141         |     delNode->next->prev = delNode->prev;

```

```

141     delNode->next->prev = delNode->prev;
142 }
143
144 // change prev only if node to be deleted is NOT the first node
145 if (delNode->prev != nullptr) {
146     delNode->prev->next = delNode->next;
147 }
148
149 delete delNode;
150 }
151
152 // =====
153 // delete from end
154 // =====
155 template <typename T>
156 void dllDeleteEnd(doubleList<T>*& currentHead) {
157     if (*currentHead == nullptr) return;
158
159     doubleList<T>* temp = *currentHead;
160
161     if (temp->next == nullptr) {
162         *currentHead = nullptr;
163         delete temp;
164         return;
165     }
166
167     while (temp->next != nullptr) {
168         temp = temp->next;
169     }
170
171     temp->prev->next = nullptr;
172     delete temp;
173 }
174
175 #endif

```

OUTPUT

```

Inserting at the head
Song_Intro Song_A Song_B Song_C
Reverse Traversal: Song_C Song_B Song_A Song_Intro
General Insertion (Inserting 'Song_X' after 'Song_B'):
Song_Intro Song_A Song_B Song_X Song_C
Reverse Traversal: Song_C Song_X Song_B Song_A Song_Intro
Inserting at the end ('Song_Z'):
Song_Intro Song_A Song_B Song_X Song_C Song_Z
Reverse Traversal: Song_Z Song_C Song_X Song_B Song_A Song_Intro
General Deletion (Deleting 'Song_X'):
Song_Intro Song_A Song_B Song_C Song_Z
Reverse Traversal: Song_Z Song_C Song_B Song_A Song_Intro
Deleting from the end:
Song_Intro Song_A Song_B Song_C
Reverse Traversal: Song_C Song_B Song_A Song_Intro

```

Summary of lessons learned

- In this HOA, I learned how to implement and manipulate doubly linked lists using C++ templates. I learned how nodes store data while maintaining connections to adjacent elements through next and prev pointers. Additionally, I learned how dynamic memory works using new to create new list nodes and delete to properly free memory when nodes are removed. Finally, I learned how forward and reverse traversal work by leveraging bidirectional pointers.

Analysis of the supplementary activity

- The supplementary activity applied these doubly linked list concepts to simulate a real-world Song Playlist. Instead of basic character data, the nodes represent song titles (std::string). By implementing functions for inserting songs (head, middle, end), removing songs, and traversing through tracks, the list models common music player features such as moving to the next track, returning to a previous track, or looping through all songs in sequence.

Concluding statement / Feedback: How well did you think you did in this activity? What are your areas for improvement?

- I think I did well in this HOA and was able to follow how nodes are created, linked, and deleted step by step. However, I still need more practice handling edge cases in pointer management, such as updating pointer-to-pointer references (**currentHead) and preventing null pointer dereferences when deleting nodes from empty or single-node lists.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Copilot

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Idea generation. Code assistance

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: I used Copilot to elaborate the thoughts I have in my mind without replacing the original thought and also for getting the formulas that are needed. I affirm that my use of AI complies with TIP's AI usage policy.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.